Structure-Aware Mathematical Reasoning for Verified Optimization in OptimAI

1 Background and Motivation

Large language models have demonstrated strong performance in natural language understanding, but they remain unreliable in mathematical reasoning, numerical stability, and algorithmic decision-making. In optimization settings, these weaknesses appear as incorrect assumptions about problem structure, inappropriate solver selection, and a lack of verifiable correctness.

OptimAI currently operates as a meta-optimizer that orchestrates solver calls and summarizes results. However, its decisions rely primarily on heuristic pattern matching rather than principled mathematical reasoning. As a result, solver choices are brittle, explanations are informal, and outputs lack guarantees.

In contrast, human experts approach optimization problems by first reasoning about structure: convexity, smoothness, constraints, scale, and noise. Only after establishing these properties do they select algorithms and verify results.

This project focuses on embedding a *single, rigorous mathematical reasoning capability* into OptimAI: **structure-aware optimization with formal verification**. Rather than attempting to cover all possible reasoning paradigms, we deliberately restrict scope to ensure correctness, implementability, and evaluability.

2 Research Objective

The objective of this project is to design and implement a mathematical reasoning module that enables OptimAI to:

- Parse optimization problems into canonical mathematical form.
- Infer structural properties with **explicit uncertainty quantification**.
- Select from a principled set of optimization algorithms based on these properties.
- Verify solutions using first-order and second-order optimality conditions.
- Revise incorrect assumptions when verification fails.

Central research question:

How can an LLM-guided system reliably infer optimization problem structure and select gradient-based algorithms whose correctness can be formally verified?

3 Scope Restriction and Design Rationale

To keep the project focused and implementable, we make the following deliberate restrictions:

- We restrict attention to **continuous optimization problems**.
- We focus on **deterministic objectives** (no stochastic noise modeling).
- We assume **differentiability**, possibly nonconvex.
- We limit solver choices to a **small, interpretable set**.
- We focus on **local optimality verification** (global optimization is out of scope except for convex problems).

This restriction allows the system to reason precisely about gradients, Hessians, and optimality conditions, rather than relying on heuristic or black-box methods.

4 Chosen Mathematical Reasoning Strategy

Among the many reasoning strategies discussed in prior drafts, this project implements exactly one:

Structure inference via symbolic and numerical differentiation, paired with verification through KKT and second-order conditions.

4.1 Why This Choice

This choice is motivated by three factors:

1. **Theoretical grounding**: Convexity, smoothness, and optimality conditions are well-defined and mathematically testable.
2. **Implementability**: Gradients, Hessians, and KKT residuals can be computed using existing tools (AD, CAS).
3. **Verifiability**: Solutions can be accepted or rejected based on formal conditions rather than heuristic confidence.

Other reasoning modes (Bayesian optimization, evolutionary methods, combinatorial solvers) are intentionally excluded, as they lack strong certificates of correctness and would dilute the core contribution.

5 System Architecture

The proposed OptimAI extension follows a minimal, linear pipeline:

5.1 1. Parsing and Canonicalization

5.1.1 Input Representation

The system accepts optimization problems in three formats:

1. **Natural language**: "Minimize the Rosenbrock function subject to $x^2 + y^2 \leq 1$"
2. **LaTeX**: Mathematical expressions parsed via regex and SymPy
3. **Python code**: Direct function definitions

5.1.2 Canonical Form

All problems are converted to the standard form:

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{s.t.} \quad g_i(x) \leq 0,\ i = 1, \ldots, m, \quad h_j(x) = 0,\ j = 1, \ldots, p$$

5.1.3 Implementation Details

Data Structure:

```
class OptimizationProblem:
    objective: SymbolicExpression    # f(x)
    variables: List[Symbol]          # [x1, x2, ..., xn]
    inequality_constraints: List[SymbolicExpression]  # [g1, g2, ...]
    equality_constraints: List[SymbolicExpression]    # [h1, h2, ...]
    bounds: Dict[Symbol, Tuple[float, float]]        # Variable bounds
    initial_point: Optional[np.ndarray]              # x0
```

Parsing Algorithm:

Validation: The canonicalization step includes:

- Syntax validation via SymPy parsing
- Dimensional consistency checks (all variables appear in at least one expression)
- Domain verification (functions are defined over $\mathbb{R}^n$)
- Differentiation test (can symbolic gradients be computed?)

Algorithm 1 Problem Parsing and Canonicalization

Require: Natural language description D
Ensure: OptimizationProblem instance P

objective_str ← LLM.extract_objective(D)
constraint_strs ← LLM.extract_constraints(D)
f ← SymPy.parse(objective_str)
vars ← SymPy.free_symbols(f)
for c in constraint_strs **do**
    expr ← SymPy.parse(c)
    if c contains "$\leq$" or "$<$" **then**
        Convert to form $g(x) \leq 0$
        Append to P.inequality_constraints
    else if c contains "$=$" **then**
        Convert to form $h(x) = 0$
        Append to P.equality_constraints
    end if
end for
Validate dimensionality and bound consistency
return P

5.2 2. Structural Inference

5.2.1 Overview

The structural inference module analyzes mathematical properties of the problem to guide algorithm selection. We compute both symbolic and numerical indicators of structure.

5.2.2 Convexity Detection

Symbolic Convexity: For objective $f(x)$, compute the Hessian:

$$H(x) = \nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

Sylvester's Criterion: For unconstrained problems, test if all leading principal minors are non-negative:

$$\begin{aligned}\Delta_1 &= H_{11} \geq 0 \\ \Delta_2 &= \det \begin{bmatrix} H_{11} & H_{12} \\ H_{21} & H_{22} \end{bmatrix} \geq 0 \\ &\vdots \\ \Delta_n &= \det(H) \geq 0\end{aligned}$$

Eigenvalue Sampling with Uncertainty: When symbolic analysis is intractable, sample N points $\{x^{(1)}, \ldots, x^{(N)}\}$ uniformly from the feasible region and compute:

$$\lambda_{\min}^{(i)} = \min \operatorname{eig}(H(x^{(i)}))$$

Define empirical convexity proportion:

$$\hat{p}_{\text{cvx}} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}\{\lambda_{\min}^{(i)} \geq -\epsilon\}$$

where $\epsilon = 10^{-6}$ is a numerical tolerance.

Uncertainty quantification: Compute Wilson score confidence interval for the proportion at 95% confidence level:

$$\text{CI}_{0.95} = \left[\frac{\hat{p} + \frac{z^2}{2N} - z\sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}}, \frac{\hat{p} + \frac{z^2}{2N} + z\sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}} \right]$$

where $z = 1.96$ for 95% confidence.

High-dimensional warning: For $n > 20$ and $N < 1000$, flag result as "UNCERTAIN due to curse of dimensionality" regardless of $\hat{p}_{\text{cvx}}$.

5.2.3 Smoothness and Lipschitz Estimation

Lipschitz Constant Estimation: For gradient Lipschitz constant L, we have:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$$

Estimate a **lower bound** $\hat{L}_{\text{lower}}$ via finite differences over sample pairs:

$$\hat{L}_{\text{lower}} = \max_{i,j} \frac{\|\nabla f(x^{(i)}) - \nabla f(x^{(j)})\|}{\|x^{(i)} - x^{(j)}\|}$$

Note: This provides only a lower bound on L. The true Lipschitz constant may be arbitrarily larger. This estimate is used only for relative comparisons between problems.

Algorithm 2 Convexity Inference

Require: Objective f, variables $\mathbf{x}$, sample size N
Ensure: Convexity assessment with uncertainty

$H \leftarrow \text{SymPy.hessian}(f, \mathbf{x})$
symbolic_convex $\leftarrow$ CheckSylvester(H)
if symbolic_convex = True **then**
    return (CONVEX, confidence=1.0, method="symbolic")
else if symbolic_convex = False **then**
    return (NONCONVEX, confidence=1.0, method="symbolic")
else ▷ Symbolic test inconclusive
    $n \leftarrow \dim(\mathbf{x})$
    if $n > 20$ and $N < 1000$ **then**
        return (UNCERTAIN, confidence=0.0, warning="curse of dimensionality")
    end if
    sample_points $\leftarrow$ SampleFeasibleRegion(N)
    psd_count $\leftarrow 0$
    for x in sample_points **do**
        $H_{\text{num}} \leftarrow \text{EvaluateHessian}(H, x)$
        $\lambda_{\min} \leftarrow \min(\text{eig}(H_{\text{num}}))$
        if $\lambda_{\min} \geq -10^{-6}$ **then**
            psd_count $\leftarrow$ psd_count $+ 1$
        end if
    end for
    $\hat{p} \leftarrow$ psd_count$/N$
    CI $\leftarrow$ WilsonScoreInterval($\hat{p}, N$)
    return (EMPIRICAL, proportion=$\hat{p}$, CI=CI, method="sampling")
end if

Condition Number: Compute the condition number of the Hessian at sampled points:

$$\kappa(x) = \frac{\lambda_{\max}(H(x))}{|\lambda_{\min}(H(x))|}$$

Report the median κ across samples. Problems with median $\kappa > 10^4$ are flagged as ill-conditioned.

5.2.4 Constraint Analysis

For each constraint $g_i(x) \leq 0$ or $h_j(x) = 0$:

- Compute Jacobian: $\nabla g_i(x)$, $\nabla h_j(x)$
- Check linear independence (for LICQ) at sample points: $\text{rank}([\nabla h_1, \ldots, \nabla h_p, \nabla g_{\mathcal{A}(x)}]) = p + |\mathcal{A}(x)|$, where $\mathcal{A}(x) = \{i : g_i(x) \geq -\epsilon\}$ is the ϵ-active set
- Detect linear constraints: $\nabla^2 g_i = 0$

LICQ failure handling: If LICQ is violated at any sample point, flag constraint qualification as "VIOLATED" and recommend penalty methods or augmented Lagrangian approaches.

5.2.5 Structure Summary

The inference module outputs a structured report:

```
class StructureReport:
    # Convexity
    convexity_status: str            # "CONVEX", "NONCONVEX", "UNCERTAIN"
    convexity_confidence: float      # 0.0 to 1.0
    convexity_method: str            # "symbolic", "sampling"
    convexity_ci: Optional[Tuple]    # Confidence interval if sampling

    # Smoothness
    lipschitz_lower_bound: float     # Lower bound estimate
    condition_number_median: float   # Median kappa
    is_smooth: bool                  # C^2 continuity

    # Constraints
    has_constraints: bool
    has_linear_constraints: bool
    licq_status: str                 # "SATISFIED", "VIOLATED", "UNKNOWN"

    # Scale
    problem_dimension: int
    problem_scale: str               # "small" (<10), "medium" (10-50), "large" (>50)
```

5.3 3. Algorithm Selection

5.3.1 Decision Logic

The algorithm selector uses a decision tree based on the structure report:

Algorithm 3 Solver Selection (Revised)

Require: StructureReport S
Ensure: Selected solver name and justification

if S.has_constraints **then**
  if S.licq_status = "VIOLATED" **then**
    return ("Augmented Lagrangian", "LICQ violated, penalty approach needed")
  else if S.has_linear_constraints only and S.convexity_status = "CONVEX" **then**
    return ("Interior Point", "Convex with linear constraints")
  else
    return ("SLSQP", "General constrained problem with LICQ")
  end if
else ▷ Unconstrained
  if S.convexity_status = "CONVEX" **then**
    if S.condition_number_median < 100 **then**
      return ("L-BFGS", "Well-conditioned convex problem")
    else
      return ("Gradient Descent", "Ill-conditioned convex, robust choice")
    end if
  else if S.convexity_status = "NONCONVEX" **then**
    if S.is_smooth **then**
      return ("Trust-Region Newton", "Nonconvex smooth, second-order method")
    else
      return ("Nelder-Mead", "Nonconvex non-smooth, derivative-free")
    end if
  else ▷ UNCERTAIN
    return ("L-BFGS with multi-start", "Uncertain structure, robust quasi-Newton + restarts")
  end if
end if

5.3.2 Solver Specifications

Gradient Descent:

$$x^{(k+1)} = x^{(k)} - \alpha_k \nabla f(x^{(k)})$$
$$\alpha_k = \text{Backtracking line search with Armijo condition}$$

Parameters: $\alpha_0 = 1.0$, $\rho = 0.5$, $c = 10^{-4}$, max_iter = 1000

L-BFGS: Limited-memory BFGS with approximate Hessian inverse:

$$x^{(k+1)} = x^{(k)} - \alpha_k H_k \nabla f(x^{(k)})$$

where H_k is constructed from the last m iterates.

Parameters: $m = 10$, line search = Strong Wolfe, max_iter = 500

Trust-Region Newton: Solve the subproblem:

$$\min_p f(x^{(k)}) + \nabla f(x^{(k)})^T p + \frac{1}{2} p^T H(x^{(k)}) p \quad \text{s.t.} \quad \|p\| \leq \Delta_k$$

using the Steihaug-Toint CG method, which handles indefinite Hessians via early termination on negative curvature.

Parameters: $\Delta_0 = 1.0$, $\eta = 0.1$ (acceptance threshold), max_iter = 500

SLSQP (Sequential Least Squares Programming): For constrained problems satisfying LICQ. Uses quadratic approximations of objective and linear approximations of constraints.

Parameters: max_iter = 500, ftol = 10^{-6}

Augmented Lagrangian: For problems with constraint qualification violations:

$$\min_x f(x) + \sum_i \mu_i g_i(x)^+ + \frac{\rho}{2} \sum_i (g_i(x)^+)^2$$

where $g^+ = \max(0, g)$ and μ_i, ρ are penalty parameters updated iteratively.

Interior Point: For convex problems with linear constraints. Maintains strict feasibility via barrier functions.

5.4 4. Execution with Automatic Differentiation

5.4.1 Gradient and Hessian Computation

We use a hybrid approach:

1. **Symbolic differentiation**: For simple expressions, compute gradients symbolically with SymPy, then lambdify.
2. **Automatic differentiation**: For complex expressions, use JAX for forward/reverse mode AD.

Symbolic Approach:

```
import sympy as sp
f_sym = sp.sympify("x**2 + y**2")
grad_sym = [sp.diff(f_sym, var) for var in [x, y]]
grad_func = sp.lambdify([x, y], grad_sym, "numpy")
```

AD Approach:

```
import jax.numpy as jnp
from jax import grad, hessian

def f(x):
    return jnp.sum(x**2)

grad_f = grad(f)
hess_f = hessian(f)
```

5.4.2 Numerical Stability

To ensure stable gradient computation:

- Normalize inputs to unit scale: $\tilde{x} = (x - \mu)/\sigma$ (track normalization parameters for denormalization)
- Use machine epsilon checks: $\|\nabla f\| < 10^{-14}$ is treated as zero
- **Problem-aware scaling**: Instead of gradient clipping (which breaks optimization mathematics), detect gradient explosion and recommend problem reformulation or scaling

5.5 5. Verification and Certification

5.5.1 First-Order Optimality (Unconstrained)

For unconstrained problems, check:

$$\|\nabla f(x^*)\| \leq \epsilon_g = 10^{-6}$$

If this fails, the solution is rejected.

5.5.2 KKT Conditions (Constrained)

For problems with constraints, verify the Karush-Kuhn-Tucker conditions:

$$\nabla f(x^*) + \sum_{i=1}^{m} \mu_i^* \nabla g_i(x^*) + \sum_{j=1}^{p} \lambda_j^* \nabla h_j(x^*) = 0 \tag{1}$$

$$g_i(x^*) \leq 0, \quad i = 1, \ldots, m \tag{2}$$

$$h_j(x^*) = 0, \quad j = 1, \ldots, p \tag{3}$$

$$\mu_i^* \geq 0, \quad i = 1, \ldots, m \tag{4}$$

$$\mu_i^* g_i(x^*) = 0, \quad i = 1, \ldots, m \tag{5}$$

Multiplier Recovery: Critical implementation detail: Lagrange multipliers μ^*, λ^* must be obtained from the solver output (SLSQP, Interior Point, and Augmented Lagrangian track these). For solvers that don't provide multipliers (L-BFGS applied to unconstrained reformulations), we:

1. Use only first-order optimality checks
2. **OR** solve the KKT stationarity condition as a least-squares problem to recover approximate multipliers:

$$\min_{\mu,\lambda} \left\| \nabla f(x^*) + \sum_{i} \mu_i \nabla g_i(x^*) + \sum_{j} \lambda_j \nabla h_j(x^*) \right\|^2$$

   subject to $\mu \geq 0$, which gives a residual-minimizing multiplier estimate.

Algorithm 4 KKT Verification

Require: Solution x^*, multipliers μ^*, λ^* (from solver or recovered)
Ensure: Pass/fail and residual norms

$r_{\text{grad}} \leftarrow \|\nabla f(x^*) + \sum \mu_i^* \nabla g_i(x^*) + \sum \lambda_j^* \nabla h_j(x^*)\|$
$r_{\text{primal}} \leftarrow \max(\max_i g_i(x^*), \max_j |h_j(x^*)|)$
$r_{\text{dual}} \leftarrow \min_i \mu_i^*$ ▷ Check non-negativity
$r_{\text{comp}} \leftarrow \max_i |\mu_i^* g_i(x^*)|$ ▷ Complementarity
if $r_{\text{grad}} < 10^{-6}$ **and** $r_{\text{primal}} < 10^{-6}$ **and** $r_{\text{dual}} \geq 0$ **and** $r_{\text{comp}} < 10^{-6}$ **then**
    return (Pass, $(r_{\text{grad}}, r_{\text{primal}}, r_{\text{dual}}, r_{\text{comp}})$)
else
    return (Fail, $(r_{\text{grad}}, r_{\text{primal}}, r_{\text{dual}}, r_{\text{comp}})$)
end if

Implementation:

5.5.3 Second-Order Sufficient Conditions

For unconstrained problems, verify:

$$\lambda_{\min}(H(x^*)) > 0$$

For constrained problems, verify positive definiteness on the tangent space $T(x^*)$ defined by active constraints:

$$p^T H(x^*)p > 0 \quad \forall p \in T(x^*), p \neq 0$$

Tangent space construction: Let $\mathcal{A}(x^*) = \{i : g_i(x^*) \geq -\epsilon\}$ be the active inequality set. Construct the nullspace basis Z of the constraint Jacobian matrix:

$$J = [\nabla h_1(x^*), \ldots, \nabla h_p(x^*), \nabla g_i(x^*) \text{ for } i \in \mathcal{A}(x^*)]^T$$

Then verify $Z^T H(x^*) Z \succ 0$ using eigenvalue decomposition.

Degenerate constraint handling: If active set identification is ambiguous (near-zero constraint values), perform verification with multiple ϵ values $(10^{-8}, 10^{-6}, 10^{-4})$ and flag if results are inconsistent.

5.5.4 Verification Failure Handling

When verification fails:

1. **Log failure mode**: Record which condition failed and by how much.
2. **Diagnose root cause**: Distinguish between:
   - Solver stalled at non-optimal point → increase iterations
   - Incorrect structure inference → re-run with tighter sampling
   - Poor initial point → multi-start strategy
   - Numerical issues → rescale problem
3. **Escalate solver**: Switch to more robust solver (e.g., SLSQP → Augmented Lagrangian).
4. **Multi-start for nonconvex**: If problem is nonconvex, run from $K = 5$ random initializations:
   $$x_0^{(k)} \sim \mathcal{U}(\text{feasible region}), \quad k = 1, \ldots, 5$$
   Return the best verified solution.

If all attempts fail, report the best candidate with:

- Explicit warning: "redWARNING: Solution did not pass verification. Residuals: ..."
- All residual norms
- Recommendation: "Consider reformulating the problem or adjusting tolerances."

6 Explanation and Auditability

6.1 Reasoning Trace Structure

OptimAI produces a JSON-formatted reasoning trace:


```
{
  "problem": {
    "objective": "x^2 + y^2",
    "constraints": ["x + y <= 1"],
    "dimension": 2
  },
  "structural_analysis": {
    "convexity_status": "CONVEX",
    "convexity_confidence": 1.0,
    "convexity_method": "symbolic",
    "lipschitz_lower_bound": 2.83,
    "condition_number_median": 1.0,
    "is_smooth": true,
    "licq_status": "SATISFIED"
  },
  "solver_selection": {
    "chosen": "Interior Point",
    "reason": "Convex with linear constraints",
    "alternatives_considered": ["SLSQP", "Augmented Lagrangian"]
  },
  "execution": {
    "iterations": 23,
    "function_evals": 45,
    "final_objective": 0.25,
    "convergence": "SUCCESS"
  },
  "verification": {
    "gradient_norm": 1.2e-7,
    "kkt_residuals": {
      "stationarity": 1.5e-8,
      "primal_feasibility": 3.2e-9,
      "dual_feasibility": 0.0,
      "complementarity": 2.1e-9
    },
    "hessian_eigenvalues": [2.0, 2.0],
    "sosc_status": "PASS",
    "overall_status": "VERIFIED"
```

```
  }
}
```

6.2 Natural Language Explanation

The system generates human-readable explanations with appropriate epistemic humility:

> Based on these properties, I selected the Interior Point method, which is specifically designed for convex problems with linear constraints and maintains strict feasibility.
>
> The solver converged in 23 iterations to objective value 0.25. I verified the solution satisfies:
>
> - KKT stationarity (residual = 1.5×10^{-8})
> - Primal feasibility (residual = 3.2×10^{-9})
> - Dual feasibility (all multipliers non-negative)
> - Complementarity (residual = 2.1×10^{-9})
> - Second-order sufficiency (projected Hessian positive definite)
>
> The solution is **verified as a local optimum**. Since the problem is convex, this is also the global optimum." *"I analyzed the problem and determined it is convex with 100% confidence based on symbolic Hessian analysis (Sylvester's criterion). The problem is smooth (twice continuously differentiable) and well-conditioned (median condition number = 1.0). The constraints are linear and satisfy LICQ.*
>
> *Based on these properties, I selected the Interior Point method, which is specifically designed for convex problems with linear constraints and maintains strict feasibility.*
>
> *The solver converged in 23 iterations to objective value 0.25. I verified the solution satisfies:*
>
> - *KKT stationarity (residual = 1.5×10^{-8})*
> - *Primal feasibility (residual = 3.2×10^{-9})*
> - *Dual feasibility (all multipliers non-negative)*
> - *Complementarity (residual = 2.1×10^{-9})*
> - *Second-order sufficiency (projected Hessian positive definite)*
>
> *The solution is **verified as a local optimum**. Since the problem is convex, this is also the global optimum."*

7 Evaluation Plan

7.1 Test Suite

7.1.1 Convex Problems

- Quadratic: $f(x) = \frac{1}{2}x^T Qx + b^T x$, $Q \succ 0$ (known global optimum)
- Least squares: $f(x) = \|Ax - b\|^2$ (known global optimum)
- Log-sum-exp: $f(x) = \log(\sum_i e^{a_i^T x + b_i})$ (verify via CVX)

7.1.2 Nonconvex Problems

- Rosenbrock: $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$ (known global minimum at $(1, 1)$)
- Rastrigin: $f(x) = 10n + \sum_i (x_i^2 - 10\cos(2\pi x_i))$ (many local minima)
- Six-hump camel: Standard global optimization benchmark (6 local minima)

7.1.3 Constrained Problems

- Linear constraints: $Ax \leq b$ (verify with linear programming solvers)
- Quadratic constraints: $x^T Qx \leq c$ (compare with IPOPT)
- Equality constraints: $h(x) = 0$ (test LICQ violations)

7.2 Metrics

1. **Structure Inference Accuracy**: For problems with known structure (convex/nonconvex), measure:
   - True positive rate (correctly identified convex)
   - True negative rate (correctly identified nonconvex)
   - Uncertainty rate (proportion flagged as UNCERTAIN)
2. **Solver Appropriateness Score**: Expert rating (1-5) of whether chosen solver is appropriate for the identified structure.
3. **Verification Success Rate**: Fraction of solver outputs that pass all verification tests.
4. **Solution Quality (Convex Problems Only)**: For convex problems with known global optima:

$$\text{Optimality Gap} = \frac{|f(x^*) - f(x^*_{\text{true}})|}{|f(x^*_{\text{true}})| + 1}$$

5. **Solution Quality (Nonconvex, Known Global Optimum)**: For problems like Rosenbrock where global optimum is known:

$$\text{Distance to Global} = \|x^* - x_{\text{global}}\|$$

   Report distribution over multiple runs with different initializations.

6. **Verification False Negative Rate**: For problems where we deliberately inject suboptimal solutions, measure fraction incorrectly passing verification.

7. **Multi-start Improvement Rate**: For nonconvex problems, measure how often multi-start finds strictly better solutions than single runs.

7.3 Baselines

1. **Fixed Solver (L-BFGS)**: Always use L-BFGS regardless of structure.

2. **Fixed Solver (SLSQP)**: Always use SLSQP for constrained, L-BFGS for unconstrained.

3. **Heuristic Selection**: Pattern match on problem keywords (e.g., "quadratic" $\rightarrow$ L-BFGS).

4. **No Verification**: Run structure-aware selection but accept all solutions without verification.

5. **SciPy Default**: Use `scipy.optimize.minimize` with `method=None` (auto-selection).

7.4 Expected Results

We hypothesize:

- Structure-aware selection will improve solver success rate by $> 15\%$ over fixed solvers on heterogeneous test suite.
- Verification will reject $> 80\%$ of deliberately injected false solutions (low false negative rate).
- Multi-start strategy will improve best objective value by $> 25\%$ on Rastrigin and Six-hump camel.
- Convexity detection will achieve $> 90\%$ accuracy on problems with $n \leq 10$ variables.
- High-dimensional problems ($n > 20$) will correctly trigger UNCERTAIN flags $> 80\%$ of the time when sampling-based inference is used.

8 Implementation Timeline

1. **Week 1-2**: Implement parsing and canonicalization module. Test on 20 hand-crafted problems.
2. **Week 3-4**: Build structural inference (convexity, smoothness tests). Implement uncertainty quantification.
3. **Week 5**: Implement algorithm selection logic. Add constrained solvers (SLSQP, Interior Point, Augmented Lagrangian).
4. **Week 6-7**: Integrate AD, implement solver wrappers with multiplier tracking.
5. **Week 8**: Build verification module (KKT, SOSC, multiplier recovery).
6. **Week 9**: Implement multi-start and failure recovery logic.
7. **Week 10**: Test suite creation (100+ problems) and baseline implementation.
8. **Week 11-12**: Run experiments, analyze results, iterate on failure cases.

9 Expected Contributions

This project contributes:

1. A narrowly scoped, mathematically rigorous reasoning module for OptimAI with explicit uncertainty quantification.
2. A concrete demonstration of verification-driven algorithm selection for both convex and nonconvex problems.
3. A framework for handling constraint qualification failures and multiplier recovery.
4. A reproducible evaluation framework distinguishing between local and global optimality verification.
5. Open-source implementation with comprehensive test suite and evaluation scripts.

Rather than attempting general intelligence, this work demonstrates how constrained mathematical reasoning with honest uncertainty reporting can make LLM-based optimization systems reliable and trustworthy.

10 Technical Challenges and Mitigation Strategies

10.1 Challenge 1: Symbolic Analysis Scalability

Problem: Computing symbolic Hessians for high-dimensional problems is intractable.
Mitigation:

- Fall back to numerical sampling when symbolic computation exceeds 5 seconds or problem dimension > 50.
- Explicitly report uncertainty and provide confidence intervals for sampling-based inference.
- Flag high-dimensional problems ($n > 20$) as UNCERTAIN when sampling is used.

10.2 Challenge 2: Numerical Instability

Problem: Ill-conditioned problems cause gradient overflow or underflow.
Mitigation:

- Input normalization with tracked scaling parameters.
- Detect gradient explosion and recommend problem reformulation rather than applying gradient clipping (which breaks optimization mathematics).
- Adaptive tolerances based on problem conditioning.

10.3 Challenge 3: Constraint Qualification Failures

Problem: KKT conditions require LICQ, which may not hold.
Mitigation:

- Detect LICQ violations early via rank checks on constraint Jacobians.
- Route to penalty methods (Augmented Lagrangian) when LICQ fails.
- Report constraint qualification status explicitly in reasoning trace.

10.4 Challenge 4: Multiplier Unavailability

Problem: Some solvers (L-BFGS, Gradient Descent) don't compute Lagrange multipliers.
Mitigation:

- Use only solvers that track multipliers for constrained problems (SLSQP, Interior Point, Augmented Lagrangian).
- If necessary, recover approximate multipliers via least-squares fit to KKT stationarity.
- For unconstrained problems, use only first-order optimality checks.

10.5 Challenge 5: Local vs. Global Optimality

Problem: Verification only certifies local optimality; nonconvex problems may have many poor local minima.

Mitigation:

- Explicitly state that verification certifies **local optimality only** for nonconvex problems.
- Implement multi-start strategy (5-10 random initializations) for nonconvex problems.
- Report best verified solution among all starts with distribution of objective values.
- Never claim global optimality for nonconvex problems unless problem has special structure (e.g., known global minimum).

10.6 Challenge 6: Curse of Dimensionality in Sampling

Problem: Uniform sampling becomes exponentially sparse in high dimensions, making convexity detection unreliable.

Mitigation:

- Implement explicit dimension checks: flag $n > 20$ with $N < 1000$ samples as UNCERTAIN.
- Provide confidence intervals using Wilson score method.
- Consider adaptive sampling strategies (e.g., sample along eigenvector directions of Hessian).
- Be transparent in reporting: "Convexity could not be reliably determined due to high dimensionality."

11 Related Work

11.1 LLM-Based Optimization

Prior work (OptimLLM, LLM4Opt) uses LLMs for problem formulation and hyperparameter tuning but lacks formal verification and explicit uncertainty quantification.

11.2 Automated Algorithm Selection

Algorithm selection frameworks (Auto-sklearn, CASH) use meta-learning but do not provide mathematical guarantees or verification certificates.

11.3 Verified Optimization

Formal methods (Coq, Isabelle) can prove solver correctness but are not integrated with LLMs and do not handle numerical optimization.

11.4 Constrained Optimization Solvers

Standard packages (SciPy, NLopt, IPOPT) provide robust solvers but lack automated structure inference and reasoning explanations.

This work bridges these areas by combining LLM-guided reasoning with formal verification, explicit uncertainty quantification, and comprehensive handling of both convex and nonconvex constrained optimization.